#9 return both the list of centroids and the list of canopies.

Each reducer will then work on a single canopy, merging the lists for that canopy produced by each mapper; the final result will be the list of canopies produced by each reducer.

This sub-routine described in lines #6-#8 is also illustrated in figure 13.11; this concludes our discussion on canopy clustering, and we encourage the reader to try to write down a job for this algorithm with one of the open source implementations of MapReduce, for instance Hadoop, or check out Mahout, a distributed linear algebra framework by Apache foundation, that does implement a distributed version of canopy clustering[16].

13.4 MapReduce DBSCAN

So far so good, our first attempt to distributed clustering was with k-means and we were in luck, we discovered that it can be easily re-written as a distributed algorithm because its steps can be performed independently for each point (classification) or each cluster (re-centering). We applied MapReduce to canopy clustering as well, even if its first step, drawing the canopies centroids, was not immediately parallelizable and required a deeper reasoning to obtain the best clustering.

To close the circle and complete the topic, in this section we are going to discuss how to apply the MapReduce paradigm to a clustering algorithm that is intrinsically non-parallelizable, at least at first glance: DBSCAN.